

Justificativa da escolha de banco de dados — Conexão Solidária

Hackathon Fase 5 — POSTECH .NET, Turma 11NETT, Grupo 30

Contexto

O Conexão Solidária é uma plataforma que conecta doadores a campanhas de arrecadação conduzidas por uma ONG, com um painel público de transparência sobre os valores captados. Por lidar com dados financeiros — o registro de cada doação e o valor total arrecadado por campanha — a escolha do banco de dados não é um detalhe de implementação: ela determina que garantias o sistema consegue oferecer sobre a consistência desses números diante de falhas, concorrência e reprocessamento de mensagens. Este documento explica a decisão tomada pelo grupo e o raciocínio por trás dela, para que possa ser avaliado sem a necessidade de consultar o código-fonte.

Por que um banco relacional (PostgreSQL)

Doações e campanhas são, na essência, registros financeiros: uma doação tem um valor e um status que precisa refletir com exatidão se ela foi efetivamente confirmada, e o valor arrecadado de uma campanha é a soma confiável dessas doações. Esse tipo de dado exige as garantias clássicas de um banco relacional com suporte a transações ACID — atomicidade, consistência, isolamento e durabilidade — porque não há espaço para "quase certeza": uma doação está confirmada ou não está, e o valor arrecadado de uma campanha precisa corresponder exatamente à soma das doações confirmadas, nunca a um valor aproximado ou temporariamente divergente. Um banco não relacional, otimizado para escala horizontal e disponibilidade em detrimento de consistência forte, resolveria um problema que o Conexão Solidária não tem — o volume de dados do MVP é modesto — e abriria mão exatamente da garantia mais importante para o domínio: a confiabilidade dos números que aparecem no painel de transparência. Por isso o grupo optou pelo PostgreSQL, um banco relacional maduro, com suporte transacional completo e amplamente compatível com o ecossistema .NET/EF Core já usado no restante do projeto.

Por que uma única instância com três schemas, em vez de três bancos separados

O sistema é organizado em três áreas de responsabilidade — Identidade (autenticação de usuários), Campanha (cadastro e acompanhamento das campanhas de arrecadação) e Doação (registro das doações recebidas) —, e cada uma delas poderia, em tese, ter seu próprio banco de dados fisicamente isolado. O grupo decidiu não seguir esse caminho. Em vez de três bancos separados, o projeto usa **uma única instância do PostgreSQL, dividida em três schemas** (identidade, campanha e doacao), o que preserva a separação lógica entre os módulos — cada um só acessa seu próprio schema no dia a dia — sem pagar o preço de ter os dados fisicamente espalhados em processos de banco distintos.

O motivo central dessa escolha está no momento em que uma doação é processada. Quando uma doação chega, dois efeitos precisam acontecer juntos, sem exceção: o valor arrecadado da campanha correspondente precisa ser incrementado, e a doação precisa ser marcada como confirmada. Se esses dois dados morassem em bancos fisicamente diferentes, garantir que os dois aconteçam — ou nenhum dos dois — exigiria coordenar duas transações independentes, o que é exatamente o problema que técnicas como transações distribuídas, protocolos de duas fases ou consistência eventual tentam resolver, cada uma com sua própria complexidade operacional. Com os dois schemas na mesma instância de PostgreSQL, o serviço responsável por processar a doação (o Worker) consegue abrir **uma única transação real do banco**, que atualiza a campanha e confirma a doação ao mesmo tempo: ou as duas mudanças são gravadas com sucesso, ou nenhuma delas é, sem risco de o sistema ficar num estado intermediário em que uma doação aparece como confirmada mas o valor da campanha ainda não foi somado, ou vice-versa.

O que essa escolha evita

A alternativa a essa transação única seria manter bancos fisicamente separados e introduzir um mecanismo para sincronizá-los com segurança — tipicamente um padrão de *outbox* (gravar a intenção de atualização numa tabela auxiliar e publicá-la de forma confiável em um processo à parte) ou uma *saga* (uma sequência de passos coordenados, cada um com sua própria lógica de compensação caso algo falhe no meio do caminho). Esses padrões existem por um bom motivo e são a escolha correta quando bancos realmente precisam ficar em instâncias ou tecnologias diferentes, mas eles adicionam uma camada real de complexidade de implementação, teste e operação. Dado o prazo do hackathon e o fato de que os três domínios do projeto convivem naturalmente bem numa única instância de banco, o grupo decidiu que essa complexidade adicional não se justificava: o schema único por instância entrega a mesma garantia de consistência com uma transação de banco comum, sem precisar reconstruir manualmente, em nível de aplicação, algo que o próprio PostgreSQL já resolve de forma nativa.

O papel da fila de mensageria mesmo com um único banco

Um banco único poderia sugerir, à primeira vista, que a fila de mensageria (RabbitMQ) deixaria de ser necessária — afinal, os dados já estão todos acessíveis no mesmo lugar. Não é esse o motivo pelo qual ela existe no projeto. A fila não serve para "conectar dois bancos diferentes"; ela serve para desacoplar o momento em que a doação é recebida via requisição HTTP do momento em que ela é efetivamente processada. Isso importa por várias razões práticas: o edital do hackathon exige explicitamente o uso de um broker de mensageria nesse fluxo, como comprovação de uma arquitetura orientada a eventos; a API consegue responder rapidamente ao doador (um "recebido, aguarde a confirmação") sem depender da conclusão da transação de banco naquele exato instante; se o processamento falhar por qualquer motivo, a mensagem permanece na fila e é reprocessada, em vez de a doação simplesmente se perder; e, em um pico de doações, é possível escalar apenas o serviço Worker que consome a fila, sem precisar escalar a API inteira. Em resumo, ter um único banco elimina a necessidade de comunicação entre serviços diferentes para manter dados consistentes, mas não elimina o valor de separar, no tempo, o recebimento da doação do seu processamento — e é exatamente esse

desacoplamento que a fila garante.

Como a idempotência é garantida

Um efeito colateral inevitável de usar uma fila de mensageria é que ela pode, em determinadas situações (uma falha de rede, um reinício do consumidor antes de confirmar o recebimento da mensagem), entregar a mesma mensagem mais de uma vez. Se o Worker simplesmente reaplicasse o incremento do valor arrecadado toda vez que recebesse uma mensagem, uma doação reentregada poderia ser contabilizada em dobro — um erro grave para um sistema que lida com dinheiro. Para evitar isso, o processamento é idempotente: antes de aplicar qualquer alteração, o Worker verifica, dentro da mesma transação, se aquela doação já consta numa tabela de controle (`doacao.doacoes_processadas`). Se já constar, a mensagem é descartada sem gerar nenhum efeito adicional; se não constar, o Worker atualiza o valor da campanha, confirma a doação e registra o identificador da doação nessa tabela de controle, tudo na mesma transação atômica descrita anteriormente. Dessa forma, quantas vezes a mensagem for reentregada, o resultado final no banco é sempre o mesmo — a doação processada exatamente uma vez.

Conclusão

A combinação de um banco relacional único (PostgreSQL) organizado em schemas por módulo, transações atômicas para as operações que precisam acontecer em conjunto, uma fila de mensageria para desacoplar recebimento de processamento e uma tabela de controle para garantir idempotência forma um desenho que entrega as garantias de consistência que um sistema financeiro exige, com um nível de complexidade compatível com o prazo de um hackathon — sem abrir mão de nenhuma dessas garantias por causa disso.